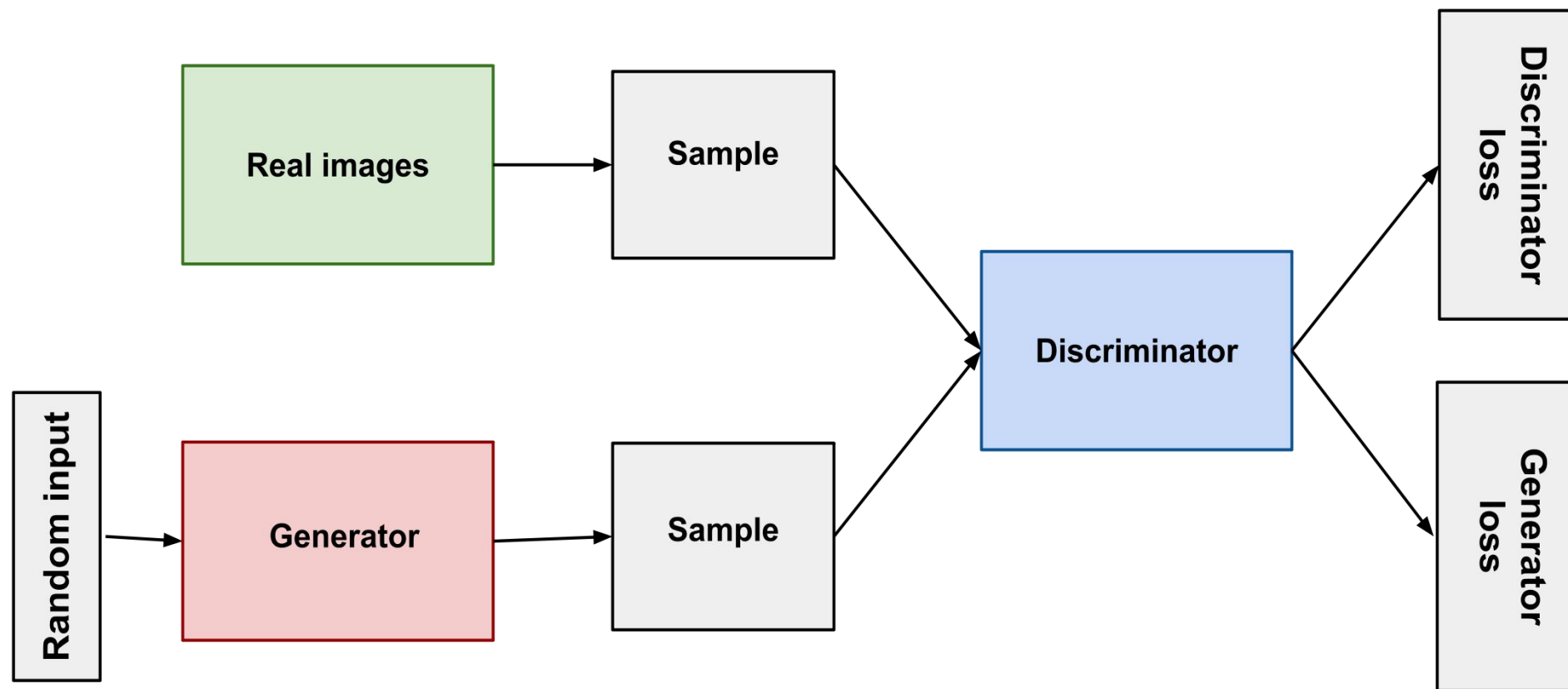
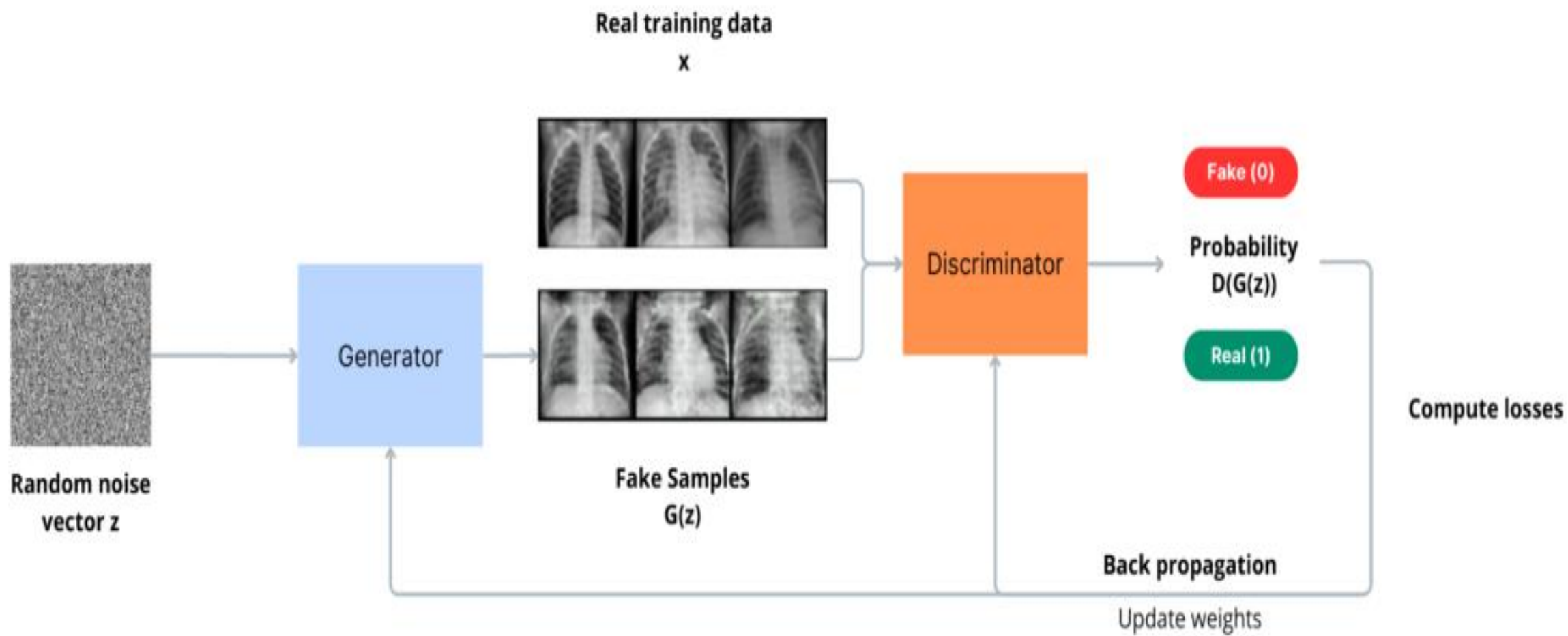
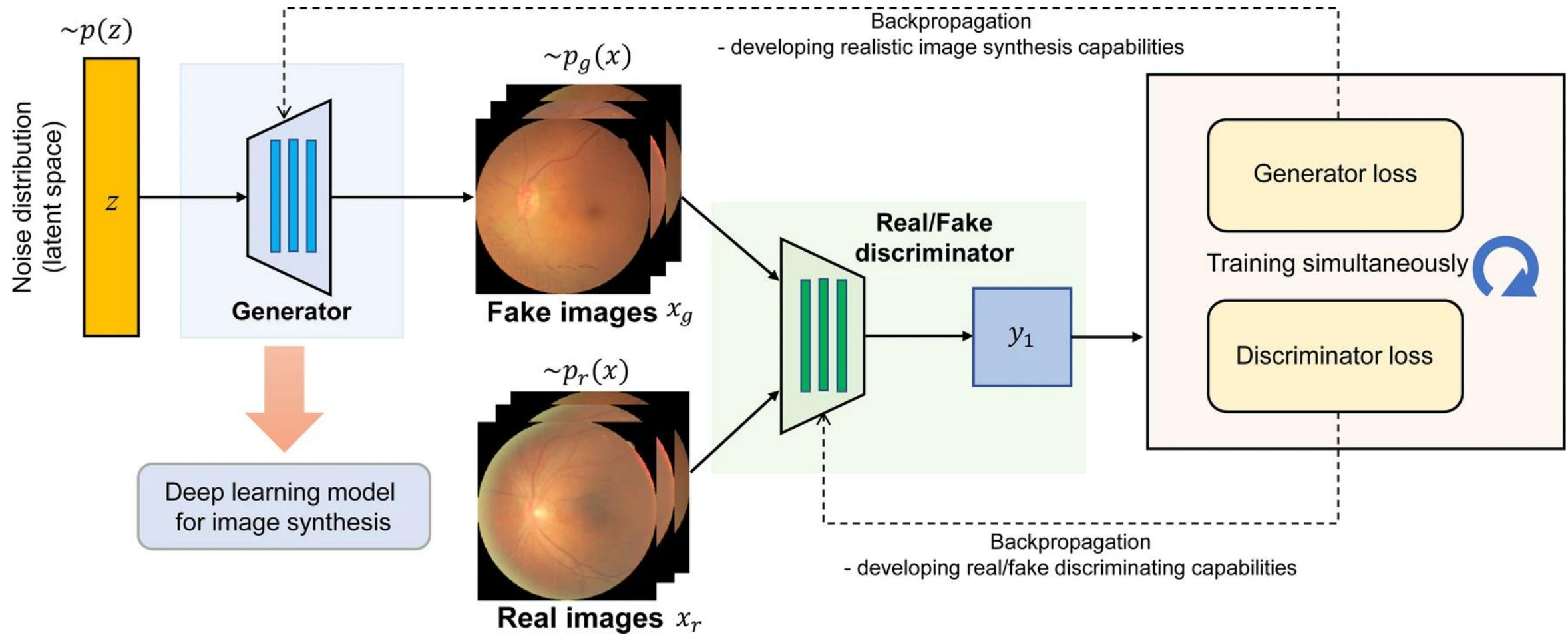


GANs in RS

Understanding GANs







When Should You Use GANs for Recommendation Noise Reduction?

Use GAN-based recommenders when you have:

Strong implicit noise

(CLICKSTREAM, WATCH LOGS, DWELL TIME)

Sparse user–item interactions

(e-commerce, video platforms)

Noisy long-tail items

(new products, rare categories)

Adversarial noise

(fake ratings or bot-generated feedback)

You want distribution-level denoising

not instance-level cleaning.

GAN-Based Recommendation Models for Mitigating the Data Noise Issue

Real-world recommender system data is inherently noisy due to:

Implicit feedback noise

Click $\neq$ interest

Skip $\neq$ dislike

Long dwell time $\neq$ guaranteed preference

User behavior randomness

Accidental clicks

Trend-driven bursts

Position bias (items shown first get more clicks regardless of relevance)

Sparse & long-tail interactions

Very few interactions for most items $\rightarrow$ sensitive to noise

Adversarial or manipulative inputs

Fake reviews

Bot-generated ratings

GANs in RS

- Traditional collaborative filtering (CF) and deep neural recommenders suffer because they try to fit the noisy data, which leads to:
- Overfitting
- Poor generalization
- Unstable ranking results

Generative Adversarial Networks (GANs) address this by learning noise-resistant user–item distributions.

Why GANs Are Suitable for Noise Mitigation

- GANs involve:
- Generator (G): learns to produce high-quality, noise-free user–item interactions.
- Discriminator (D): distinguishes true interactions vs. generated (clean) ones.
- Instead of learning from raw noisy data, G learns the underlying true preference distribution, while D filters noise.

Comparision

Noise Type	Traditional CF	GAN Models	Gain
Implicit feedback noise	Weak	Strong	+6% to +18% NDCG
Sparse data	Very weak	Moderate	+10% to +25% Recall@K
Noisy long-tail	Weak	Strong	+20% to +34%
Adversarial/Fake data	Fails completely	Maintains stability	Large

Limitations & Challenges

GANs are powerful, but not perfect:

Training instability

GAN training requires careful balancing.

Convergence issues

May collapse to recommending only a few items.

Computational overhead

More expensive than MF/CF or LightGCN.

Difficulty in handling structured noise

E.g., when noisy feedback has temporal patterns.

Scalability concerns

In huge recommendation datasets (millions of users).
BUT this is being addressed with:

- Wasserstein GANs
- Contrastive GANs
- Reinforcement-enhanced GANs

GANs enforce:

- **Distribution learning instead of instance learning**

Better resistance to random fluctuations and noise.

- **Implicit denoising mechanism**

The discriminator naturally pushes generator toward “cleaner” patterns.

- **Robustness to sparse & corrupted input**

Generator fills in missing or unreliable user–item signals.

CFGAN (Collaborative Filtering GAN)

A GAN framework specifically designed for CF systems.

How It Reduces Noise

Learns latent user preference distribution using generator.

- Discriminator learns to reject random or inconsistent feedback.
- Effectively removes “fake positives” from implicit datasets.

Why It Works

Generator learns denoised representation vectors, which are less biased and smoother.

Collaborative GAN (CGAN)

- Collaborative GAN (CGAN) is one of the earliest and most influential GAN-based frameworks designed specifically for collaborative filtering and implicit-feedback recommendation tasks, where user–item interactions are extremely sparse and often noisy.
- CGAN aims to learn the true user–item preference distribution by modeling:
 - A generator that predicts missing interactions (potential preferences)
 - A discriminator that distinguishes real user–item interactions from generated (fake) ones
- This adversarial learning prevents overfitting to noisy implicit feedback and improves robustness.

Mathematical Formulation

We can define:

Users set $U = \{1, 2, 3, 4 \dots M\}$

Item set $I = \{1, 2, 3, 4 \dots N\}$

Implicit feedback matrix

$$R \in \{0, 1\}^{M \times N}$$

where, $R_{ui} = 1$ user interacted with the item

$R_{ui} = 0$ missing interaction

Goal: Predict missing preferences despite of sparsity and randomness

CGAN Core Architecture

- CGAN consists of two modules:

(A) Generator G

Generates a preference probability score for each user–item pair:

$$G(u, i) = P(i|u; \theta_G)$$

Implementation:

- User embedding: $\mathbf{p}_u \in \mathbb{R}^d$
- Item embedding: $\mathbf{q}_i \in \mathbb{R}^d$

Score:

$$\hat{r}_{ui} = G(u, i) = \sigma(\mathbf{p}_u^\top \mathbf{q}_i)$$

The generator tries to produce item that the discriminator would classify real

CGAN Core Architecture

(B) Discriminator D

Binary classifier evaluating whether (u, i) is an actual interaction:

Binary classifier evaluating whether (u, i) is an actual interaction:

$$D(u, i) = P(R_{ui} = 1 | u, i; \theta_D)$$

Typically:

$$D(u, i) = \sigma(\mathbf{w}^\top [\mathbf{p}_u \oplus \mathbf{q}_i])$$

where $\oplus$ is concatenation.

CGAN Training Objective

CGAN inherits GAN minimax training:

- 3.1 Discriminator Loss

Discriminator maximizes the probability of:

- real interactions $\rightarrow$ labeled 1
- generated (fake) interactions $\rightarrow$ labeled 0

$$\mathcal{L}_D = - \sum_{(u,i) \in R^+} \log D(u, i) - \sum_{(u, \hat{i}) \in R^-} \log(1 - D(u, \hat{i}))$$

Where:

- R^+ real observed interactions
- R^- generated fake samples $\hat{i} \sim G(u, \cdot)$

3.2 Generator Loss

Generator tries to fool discriminator:

$$\mathcal{L}_G = - \sum_{(u, \hat{i})} \log D(u, \hat{i})$$

Meaning:

Generator success = discriminator thinks generated interaction is "real".

3.3 Minimax CGAN Objective

$$\min_{\theta_G} \max_{\theta_D} \mathcal{V}(G, D) = \mathbb{E}_{(u, i) \sim R^+} [\log D(u, i)] + \mathbb{E}_{(u, \hat{i}) \sim G} [\log(1 - D(u, \hat{i}))]$$

This adversarial mini-max game forces:

- **G** → generate realistic user-item preferences
- **D** → filter out noise and inconsistencies

How CGAN Mitigates Noise

Noisy implicit feedback includes:

1. Random clicks
2. Accidental interactions
3. Position bias
4. High sparsity
5. Non-preference signals

CGAN handles this because:

The discriminator learns to detect consistent (u, i) patterns

noisy interactions contradict learned collaborative signals → D penalizes them.

The generator learns latent preference structure

averages out noise

smooths user–item representation

GAN training discourages memorization of noisy labels

promoting distribution-level learning, not instance-level fitting.

Training Procedure Summary

For every batch:

Step 1 — Discriminator training

- Take real positives $(u, i) \in R^+$
- Generate fake samples via $G(u, \cdot)$
- Update D to classify real = 1, fake = 0

Step 2 — Generator training

- Generate fake items
- Update G to maximize discriminator's prediction of "real"

Alternate until convergence.

Limitations

Training instability

- Requires tuning learning rates and balancing G/D updates.

Sensitive to negative sampling strategy

- Quality of fake samples affects training dynamics.

Harder to scale than MF or LightGCN

- Especially for very large item sets.

EMBED_DIM = 32

NOISE_DIM = 16

```
class Generator(nn.Module):
```

```
    def __init__(self, num_users, num_items, embed_dim, noise_dim):
```

```
        super().__init__()
```

```
        self.user_emb = nn.Embedding(num_users, embed_dim)
```

```
        self.item_emb = nn.Embedding(num_items, embed_dim)
```

```
        self.fc = nn.Sequential(
```

```
            nn.Linear(embed_dim + noise_dim, 64),
```

```
            nn.ReLU(),
```

```
            nn.Linear(64, num_items),
```

```
        )
```

```
    def forward(self, user_ids):
```

```
        batch = user_ids.shape[0]
```

```
        u = self.user_emb(user_ids)                # (B, 32)
```

```
        z = torch.randn(batch, NOISE_DIM).to(device) # (B, 16)
```

```
        x = torch.cat([u, z], dim=1)
```

```
        scores = self.fc(x)                        # raw item logits
```

```
        return scores
```

```
class Discriminator(nn.Module):
```

```
    def __init__(self, num_users, num_items, embed_dim):
```

```
        super().__init__()
```

```
        self.user_emb = nn.Embedding(num_users, embed_dim)
```

```
        self.item_emb = nn.Embedding(num_items, embed_dim)
```

```
        self.fc = nn.Sequential(
```

```
            nn.Linear(embed_dim * 2, 64),
```

```
            nn.LeakyReLU(0.2),
```

```
            nn.Linear(64, 1),
```

```
            nn.Sigmoid()
```

```
        )
```

```
    def forward(self, user_ids, item_ids):
```

```
        u = self.user_emb(user_ids)
```

```
        i = self.item_emb(item_ids)
```

```
        x = torch.cat([u, i], dim=1)
```

```
        return self.fc(x)
```

```

for epoch in range(1, EPOCHS+1):
    epoch_loss_D = 0
    epoch_loss_G = 0
    for _ in tqdm(range(200), desc=f"Epoch {epoch}"):
        # ---- Train Discriminator ----
        opt_D.zero_grad()
        u, i, lbl = generate_training_batch()
        # real pairs
        pred_real = D(u, i)
        # fake item samples generated by G
        fake_scores = G(u)
        fake_items = torch.argmax(fake_scores, dim=1)
        pred_fake = D(u, fake_items.detach())
        loss_real = bce(pred_real, lbl.view(-1,1))
        loss_fake = bce(pred_fake, torch.zeros_like(pred_fake))
        loss_D = loss_real + loss_fake
        loss_D.backward()
        opt_D.step()
        # ---- Train Generator ----
        opt_G.zero_grad()
        fake_scores = G(u)
        fake_items = torch.argmax(fake_scores, dim=1)
        pred = D(u, fake_items)
        loss_G = bce(pred, torch.ones_like(pred)) # fool

```

```

discriminator
    loss_G.backward()
    opt_G.step()
    epoch_loss_D += loss_D.item()
    epoch_loss_G += loss_G.item()

```